



Compiladores

Material 3

BNF

“Forma de Backus-Naur.”

Prof. Roger Rocha Ferreira



MineonScript - Operadores Básicos

=	atribuição
==	comparação de igualdade
!	negação lógica
<	menor que
>	maior que
+	soma
-	subtração
*	multiplicação
/	divisão



MineonScript - Estruturas de Controle

se	condicional
senao	alternativa
enquanto	laço de repetição



MineonScript - Instruções Essenciais

print	saída de texto
input	entrada de dados
retorno	encerra o programa com um número ao SO

BNF (Backus–Naur Form)

O **Backus–Naur Form (BNF)** foi criado em **1959** pelo cientista da computação americano **John Backus** e aprimorado em **1960** pelo dinamarquês **Peter Naur**.

A notação surgiu para definir a sintaxe da linguagem de programação **ALGOL 60**, servindo como uma gramática formal para descrever a estrutura de linguagens de computador sem ambiguidades.

Cada linha define uma regra.

O que está entre < > é um nome de **categoria**.

O que aparece em aspas " " é texto **literal** no código.

O símbolo | significa “ou”.

As chaves { } significam “pode repetir várias vezes”.



BNF (Backus–Naur Form)

`<programa> ::= <instrucoes> <retorno>`

`<instrucoes> ::= <instrucao> | <instrucao> <instrucoes>`

`<instrucao> ::= <atribuicao>
 | <saida>
 | <entrada>
 | <condicional>
 | <loop>`

`<atribuicao> ::= <identificador> "=" <expressao>`

`<saida> ::= "print" "(" <expressao> ")"`

`<entrada> ::= <identificador> "=" "input" "(" <string> ")"`

`<condicional> ::= "se" <condicao> "{" <instrucoes> "}"
 | "se" <condicao> "{" <instrucoes> "}" "senao" "{" <instrucoes>
 "}"`

`<loop> ::= "enquanto" <condicao> "{" <instrucoes> "}"`



BNF (Backus–Naur Form)

```
<condicao> ::= <expressao> <operador_relacional> <expressao>  
            | "!" <condicao>
```

```
<operador_relacional> ::= "==" | "<" | ">"
```

```
<expressao> ::= <termo>  
              | <expressao> "+" <termo>  
              | <expressao> "-" <termo>
```

```
<termo> ::= <fator>  
          | <termo> "*" <fator>  
          | <termo> "/" <fator>
```

```
<fator> ::= <numero>  
          | <identificador>  
          | "(" <expressao> ")"
```

```
<retorno> ::= "retorna" <numero>
```

```
<identificador> ::= letra {letra | digito}
```

```
<numero> ::= digito {digito}
```



Exercício

Escreva o BNF para a calculadora KK-102

Operações:

soma

subtração

multiplicação

divisão

exponenciação

raiz quadrada

média

área do retângulo

área do triângulo

área do círculo



Proposta de Solução

Escreva o BNF para a calculadora KK-102

`<programa> ::= <expressao>`

`<expressao> ::= <operacao_binaria>`

`| <operacao_unaria>`

`| <funcao>`

`<operacao_binaria> ::= <expressao> "+" <expressao>`

`| <expressao> "-" <expressao>`

`| <expressao> "*" <expressao>`

`| <expressao> "/" <expressao>`

`| <expressao> "^" <expressao>`

`<operacao_unaria> ::= "raiz" "(" <expressao> ")"`

`<funcao> ::= "media" "(" <lista_numeros> ")"`

`| "area_retangulo" "(" <expressao> "," <expressao> ")"`

`| "area_triangulo" "(" <expressao> "," <expressao> ")"`

`| "area_circulo" "(" <expressao> ")"`

`<lista_numeros> ::= <expressao> | <expressao> "," <lista_numeros>`

LEXER / TOKENIZER

```
import re # regular expressions
import sys #Sistema Operacional

# Definição dos tipos de tokens e seus padrões.
TOKENS_FONTE = [
    ("COMENTARIO", r"//.*"), #onde '.' sign. qualquer caracter com excessão de
    '\n'.
    ("ESPACO", r"\s+"), #onde '\s' sign. ' ',\t,\n ou \r.
    ("SE", r"\bse\b"),
    ("SENAO", r"\bsenao\b"),
    ("ENQUANTO", r"\benquanto\b"),
    ("ESCREVA", r"\bescreva\b"),
    ("LEIA", r"\bleia\b"),
    ("RETORNO", r"\bretorna\b"),
    ("IGUALDADE", r"=="),
    ("IGUAL", r"="),
    ("MAIOR", r">"),
    ("MENOR", r"<"),
    ("NEGACAO", r"!"),
    ("SOMA", r"\+"),
    ("SUB", r"-"),
```



Exercício

Escreva na linguagem MineonScript

A máquina de refrigerante.

Preço: R\$ 4,50.

C

Crie um código que obtenha quanto dinheiro a pessoa inseriu. Calcule o troco (se o valor for maior que R\$ 4,50)

Se o valor for igual, exiba "Beba com moderação!"

Se o valor for menor, exiba "Dinheiro insuficiente!
Faltam R\$ X"